THERAPY REHABILITATION MANAGEMENT SYSTEM

Project Report

Submitted by:

[Rishabh Beniwal]

[2502050118]

Under the supervision of:

[Dr. Rishi Gupta]

Department of Computer Science and Engineering

[Manipal University Jaipur]

Dehmi Kalan, Near GVK Toll Plaza, Jaipur-Ajmer Expressway,

Jaipur, Rajasthan 303007

Abstract

The Therapy Rehabilitation Management System is a database-focused project designed to organize information required for patient rehabilitation services. Its relational database models patients, therapists, appointments, therapy plans, therapy sessions, exercises, session-level exercise assignments, progress measurements, medical records, bills, and bill items. MySQL is used to define structured tables, primary and foreign key relationships, views, stored procedures, stored functions, and triggers.

The implemented database contains 11 base tables and 6 views. Seven stored procedures support reusable retrieval tasks; five stored functions provide reusable calculations and counts; and two BEFORE INSERT triggers validate progress measurements and appointment status. Testing included successful retrievals and deliberate invalid inserts to confirm that selected invalid values are rejected.

The project emphasizes relational modelling, referential integrity, reusable SQL logic, and data validation. The current deliverable focuses on the database/backend layer; a graphical frontend is outside the present scope. The SQL dump is included as a reproducible backup and should be restored in a separate test database before final evaluation.

Keywords: MySQL, relational database, rehabilitation management, stored procedures, stored functions, triggers, views.

Contents

Abstract

List of Figures

List of Tables

1 Introduction: overview, objectives, relevance, scope and limitations

2 Literature Review: rehabilitation information systems and technologies

3 System Design: database overview, ER relationships, tables and views

4 Database Implementation and Testing: procedures, functions, triggers, query tests

5 Results and Discussion

6 Conclusion

7 Future Work

References

List of Figures

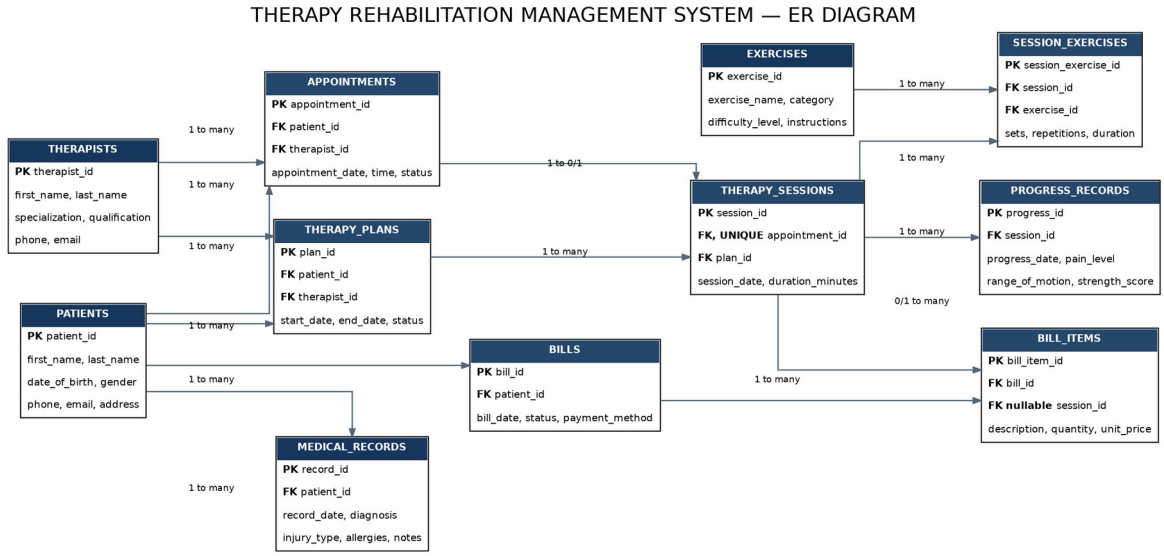


Figure 3.1: Entity-Relationship Diagram of the Therapy Rehabilitation Management System

- Figure 4.1 Invalid appointment status rejected by trigger
- Figure 4.2 Invalid pain/strength values rejected by trigger

List of Tables

- Tables 3.1–3.11: Base-table schema summaries
- Table 3.12: View descriptions
- Table 4.1: Stored procedures
- Table 4.2: Stored functions
- Table 4.3: Trigger validation tests

Chapter 1 — Introduction

1.1 Project Overview

Rehabilitation services require coordinated information about patients, therapists, appointments, treatment plans, sessions, prescribed exercises, clinical observations, and billing. When these records are disconnected or duplicated, consistency and retrieval become more difficult. The Therapy Rehabilitation Management System models these activities in a relational database implemented in MySQL.

Patient and therapist information is stored separately from appointments and treatment activities. Foreign keys connect related records, views provide joined representations, stored procedures encapsulate common retrieval tasks, functions provide reusable calculated values, and triggers validate selected inputs before insertion.

1.2 Objectives of the Project

- Design a relational schema for rehabilitation management entities.
- Represent relationships using primary keys and foreign keys.
- Reduce unnecessary duplication by separating patient, provider, appointment, treatment, clinical, and billing data.
- Implement SQL queries and views for common information needs.
- Create stored procedures for reusable parameterized operations.
- Create stored functions for calculations and counts.
- Use triggers to reject invalid appointment statuses and out-of-range progress measurements.
- Prepare a portable database backup and documentation.

1.3 Importance and Relevance

A rehabilitation database must connect clinical and administrative information while preserving relationships among records. A normalized relational design helps keep patient details in one place and allows appointments, plans, sessions, progress entries, and bills to reference stable identifiers. Constraints and triggers provide checks close to the data.

1.4 Scope and Limitations

The current scope is the database/backend layer: 11 tables, 6 views, 7 stored procedures, 5 stored functions, and 2 triggers. The implementation does not include a demonstrated graphical frontend, authentication workflow, external payment gateway, or automated clinical decision support.

The therapy_sessions table references both an appointment and a therapy plan. Although both references are constrained by foreign keys, the schema does not itself enforce that the plan and appointment belong to the same patient. This is a known business-rule limitation.

Chapter 2 — Literature Review

2.1 Rehabilitation Information Systems

Healthcare and rehabilitation information systems commonly organize patient identity, provider information, scheduled visits, care plans, interventions, observations, and administrative records. A relational database is suitable where entities have clear identifiers and relationships and consistent retrieval is required.

This is a learning-scale database model, not a certified clinical record platform. Real deployments would require additional safeguards, access control, auditability, privacy governance, and validation by qualified domain professionals.

2.2 Relevant Technologies

2.2.1 MySQL RDBMS

MySQL is the relational database management system used in this project. Primary keys identify rows; foreign keys link related rows and help maintain referential integrity. Views provide named query definitions, while stored routines and triggers implement reusable logic within the database.

2.2.2 SQL Queries and Views

JOIN operations combine related rows from multiple tables. Aggregate functions such as COUNT and SUM summarize records. Views provide named query definitions that simplify repeated joins and calculations without creating a separate copy of underlying data.

2.2.3 Stored Procedures, Functions, and Triggers

Stored procedures are invoked explicitly, commonly with CALL, and may return result sets or output parameters. Stored functions return a value and can be used in SQL expressions. Triggers execute automatically in response to configured table events. This project uses MySQL stored-program syntax; MySQL stored programs are not Oracle PL/SQL.

Chapter 3 — System Design

3.1 Database Design Overview

The database contains 11 base tables grouped into patient/provider management, appointment and therapy delivery, clinical tracking, and billing. Six views expose useful joined or summarized information. Foreign keys connect dependent records to parent entities.

3.2 Entity-Relationship Model

The relationships below are based on foreign keys recorded in the implemented schema:

- patients (1) to (many) appointments; therapists (1) to (many) appointments.
- patients (1) to (many) therapy_plans; therapists (1) to (many) therapy_plans.
- appointments (1) to (0 or 1) therapy_sessions because therapy_sessions.appointment_id is unique.
- therapy_plans (1) to (many) therapy_sessions.
- therapy_sessions (1) to (many) progress_records.
- therapy_sessions and exercises are linked through session_exercises.
- patients (1) to (many) medical_records and patients (1) to (many) bills.
- bills (1) to (many) bill_items; a bill item may optionally reference a therapy session.

Figure 3.1. Insert a final graphical ER diagram here, showing entities, primary keys, foreign keys, and cardinalities.

3.3 Description of Tables and Attributes

Schema summaries below reflect the known project definitions. Where the exact data type or nullability was not confirmed in the available project notes, the report marks it for verification against the SQL dump rather than guessing.

Table 3.1 — patients

Schema summary	Purpose
patient_id INT PK AI; first_name VARCHAR(50) NOT NULL; last_name VARCHAR(50) NOT NULL; date_of_birth DATE NOT NULL; gender VARCHAR(20); phone VARCHAR(15); email VARCHAR(100); address VARCHAR(255); emergency_contact VARCHAR(100).	Stores patient identity and contact details.

Table 3.2 — therapists

Schema summary	Purpose
therapist_id INT PK AI; first_name VARCHAR(50) NOT NULL; last_name VARCHAR(50) NOT NULL; specialization VARCHAR(100) NOT NULL; qualification, phone, email (verify exact types in dump).	Stores therapist details and specialization.

Table 3.3 — appointments

Schema summary	Purpose
appointment_id INT PK AI; patient_id INT FK NOT NULL; therapist_id INT FK NOT NULL; appointment_date DATE NOT NULL; appointment_time TIME NOT NULL; status VARCHAR(20) DEFAULT 'Scheduled'; reason VARCHAR(255).	Stores appointments and links patients to therapists.

Table 3.4 — therapy_plans

Schema summary	Purpose
plan_id INT PK AI; patient_id INT FK NOT NULL; therapist_id INT FK NOT NULL; start_date DATE NOT NULL; end_date DATE; goals TEXT; status DEFAULT 'Active'; notes TEXT (verify exact status type).	Stores rehabilitation goals and plans.

Table 3.5 — therapy_sessions

Schema summary	Purpose
session_id INT PK AI; appointment_id INT FK UNIQUE NOT NULL; plan_id INT FK NOT NULL; session_date DATE NOT NULL; duration_minutes INT; notes TEXT.	Stores sessions linked to appointments and plans.

Table 3.6 — exercises

Schema summary	Purpose
exercise_id INT PK AI; exercise_name VARCHAR(100) NOT NULL; description TEXT; category VARCHAR(50); instructions TEXT; difficulty_level VARCHAR(20).	Stores reusable exercise definitions.

Table 3.7 — session_exercises

Schema summary	Purpose
session_exercise_id INT PK AI; session_id INT FK NOT NULL; exercise_id INT FK NOT NULL; sets INT; repetitions INT; duration_minutes INT; notes TEXT.	Associates exercises with individual sessions and stores prescription details.

Table 3.8 — progress_records

Schema summary	Purpose
progress_id INT PK AI; session_id INT FK NOT NULL; progress_date DATE NOT NULL; pain_level INT; range_of_motion VARCHAR(100); strength_score DECIMAL(5,2); notes TEXT.	Stores progress measurements and observations.

Table 3.9 — medical_records

Schema summary	Purpose
record_id INT PK AI; patient_id INT FK NOT NULL; diagnosis, injury_type, treatment_history, allergies, notes; record_date DATE NOT NULL (verify exact types in dump).	Stores medical history and clinical notes.

Table 3.10 — bills

Schema summary	Purpose
bill_id INT PK AI; patient_id INT FK NOT NULL; bill_date DATE NOT NULL; status DEFAULT 'Unpaid'; payment_method VARCHAR(30) (verify exact status type).	Stores patient billing records.

Table 3.11 — bill_items

Schema summary	Purpose
bill_item_id INT PK AI; bill_id INT FK NOT NULL; session_id INT FK nullable; description VARCHAR(255) NOT NULL; quantity INT NOT NULL; unit_price DECIMAL(10,2) NOT NULL.	Stores individual bill charges.

3.4 Views

View	Purpose
appointment_details	Combines appointment data with patient and therapist details.
bill_summary	Summarizes bill items using quantity multiplied by unit price.
medical_record_details	Presents medical records with patient details.
patient_progress_history	Presents patient progress history.
session_exercise_details	Combines session and exercise details.
therapy_plan_details	Presents therapy plans with patient/therapist information.

Views are named query definitions, not independent copies of the base rows.

Chapter 4 — Database Implementation and Testing

4.1 Implementation Overview

The schema was implemented in MySQL, populated with sample records, and extended with views, stored procedures, stored functions, and triggers. The SQL backup was generated with mysqldump options to include routines, triggers, and events. The saved dump is approximately 52 KB and visibly contains table, view, routine, trigger, and data statements. A clean restore test remains recommended.

4.2 Stored Procedures

Procedure	Purpose
GetPatientAppointments(IN p_patient_id INT)	Returns appointments for a patient.
GetPatientTherapyPlans(IN p_patient_id INT)	Returns therapy plans for a patient.
GetTherapistAppointments(IN p_therapist_id INT)	Returns appointments assigned to a therapist.
GetPatientBillingTotal(IN p_patient_id INT)	Returns a patient billing total.
GetPatientProgressHistory(IN p_patient_id INT)	Returns patient progress history.
GetSessionExerciseDetails(IN p_session_id INT)	Returns exercise details for a session.
GetUnpaidBills()	Returns unpaid bills and associated details.

Procedures are explicitly invoked with CALL. Recorded examples include GetPatientAppointments(1), which returned appointments for patient 1, and GetUnpaidBills(), which returned unpaid bills in the sample data.

4.3 Stored Functions

Function	Purpose	Recorded test
CalculateBillTotal(p_bill_id INT)	Sum of quantity × unit_price for one bill.	Bill 1: 1300.00
CalculatePatientBillingTotal(p_patient_id INT)	Total across a patient's bills.	Patient 1: 2400.00
CountTherapistAppointments(p_therapist_id INT)	Counts therapist appointments.	Therapist 1: 4
CountPatientSessions(p_patient_id INT)	Counts a patient's sessions.	Patient 1: 2
CountPatientProgressRecords(p_patient_id INT)	Counts a patient's progress records.	Patient 1: 2

Functions return values and can be used in SELECT expressions. The listed outputs are recorded results from the sample dataset.

4.4 Triggers and Validation

Trigger	Event	Rule
validate_pain_level	BEFORE INSERT on progress_records	Rejects non-NULL pain_level or strength_score outside 0–10.
validate_appointment_status	BEFORE INSERT on appointments	Rejects non-NULL status values outside Scheduled, Completed, Cancelled.

4.4.1 Trigger Test Results

Test	Input	Observed result
Pain level	pain_level = 11	Rejected with SQLSTATE 45000; message specifies 0–10.
Strength score	strength_score = 11.00	Rejected with SQLSTATE 45000; message specifies 0–10.
Appointment status	status = 'Banana'	Rejected with SQLSTATE 45000; message: Invalid appointment status.
Appointment count	Count after invalid status attempt	Remained 12.

The triggers were tested against INSERT operations. The current trigger definitions do not validate later UPDATE operations.

4.5 Query and View Testing

- A JOIN query combining appointments with patient and therapist names returned 12 appointment rows in the recorded test.
- A query filtering Scheduled appointments returned appointment IDs 7, 8, 11, and 12 in the recorded test.
- Appointments grouped by therapist: therapist 1 had 4, therapist 2 had 3, therapist 3 had 3, therapist 4 had 2.
- Bill totals were calculated using SUM(quantity × unit_price).
- A subquery identified patients with completed appointments.
- The session_exercise_details view returned 21 rows in the recorded test; therapy_plan_details returned six plans.

These are recorded test results. Before final submission, restore the dump in a separate test database and repeat representative checks.

Chapter 5 — Results and Discussion

5.1 Presentation of Results

The project produced a MySQL database containing 11 base tables, 6 views, 7 stored procedures, 5 stored functions, and 2 triggers. It models patient and therapist information, appointments, treatment plans and sessions, exercises, clinical tracking, and billing.

- Foreign keys link related entities and help preserve referential integrity.
- Views simplify repeated joins and summaries.
- Stored procedures package reusable parameterized retrieval tasks.
- Stored functions centralize billing calculations and entity counts.
- Triggers reject the tested invalid progress values and appointment status.

5.2 Discussion of Findings

The design separates entities to avoid repeating details unnecessarily. For example, appointments store patient_id and therapist_id rather than duplicating names; names can be retrieved through joins. Bill items store individual charge lines and allow totals to be calculated from quantity and unit price.

The UNIQUE constraint on therapy_sessions.appointment_id enforces at most one session record per appointment. This is a design choice, not a universal rehabilitation rule. In addition, separate foreign keys do not guarantee that the plan and appointment referenced by a session belong to the same patient.

5.3 Comparison with Expected Outcomes

Core backend goals—relational schema, query and view implementation, stored routines, and trigger-based validation—have been implemented and tested with example operations. The project is a database-focused prototype, not a complete clinical application. A clean restore and repeatable regression tests are the main remaining verification tasks.

Chapter 6 — Conclusion

6.1 Summary of Project Work

The Therapy Rehabilitation Management System was implemented as a MySQL database project with 11 tables, 6 views, 7 stored procedures, 5 stored functions, and 2 triggers. Sample data and SQL queries demonstrate joins, filters, grouping, subqueries, billing calculations, and record retrieval.

6.2 Achievement of Objectives

The database models key rehabilitation management activities and applies primary keys, foreign keys, uniqueness constraints, views, stored routines, and triggers. Trigger tests demonstrated rejection of selected invalid values. A SQL dump provides a portable representation of the schema, sample data, and stored objects.

6.3 Lessons Learned

The project reinforced the importance of defining entities and relationships before writing queries. Foreign keys maintain links, views simplify repeated queries, procedures package parameterized tasks, functions return calculated values, and triggers validate data automatically at configured events.

Chapter 7 — Future Work

7.1 Suggestions for Future Enhancements

- Develop a graphical interface for authorized users.
- Add role-based access control and audit trails appropriate for sensitive clinical data.
- Review appointment/session cardinality and enforce patient-plan consistency across linked records.
- Extend validation to relevant UPDATE operations.
- Create automated test scripts to run after restoring the database.
- Add date-range reporting for progress, therapist workload, unpaid bills, and plan outcomes.
- Establish privacy, backup-retention, and access policies before using real patient data.

7.2 Areas for Further Research

Further work could evaluate indexing and query performance as the data grows. Transaction handling, access control, backup restoration, and data minimization are also important. Any clinical scoring or recommendations would require domain review and validation.

References

1. MySQL. (n.d.). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
2. MySQL. (n.d.). Stored Objects. https://dev.mysql.com/doc/refman/8.0/en/stored-objects.html
3. MySQL. (n.d.). CREATE TRIGGER Statement. https://dev.mysql.com/doc/refman/8.0/en/create-trigger.html
4. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.
5. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.